

Tehničko veleučilište u Zagrebu

Stručni studij računarstva

**IZRADA SPRING BOOT API-a PRIMJENOM RAZLIČITIH
VRSTA KOMUNIKACIJE S BAZOM PODATAKA**

Završni rad

Mentor: Jakob Gračanin, univ. mag. ing. comp., v. pred.

Student: Luka Šarić

Zagreb, rujan 2025.

Sažetak

U ovom završnom radu uz praktični primjer Spring Boot API-a uspoređuje se 3 različita načina komunikacije s bazom podataka: Spring JDBC, Hiberante ORM i MyBatis. Cilj je prikazati i analizirati prednosti i nedostatke svake tehnologije kako bi budući projekti pravilno znali odrediti koju tehnologiju koristiti.

Rad također opisuje i sve ostale tehnologije i alate koji su bili korišteni kako bi se postigli rezultati, kao: PostgreSQL, Java, Spring Boot, OpenAPI. Za praktičku usporedbu je napravljena aplikacija ljudskih resursa kroz koju se može upravljati radnicima, odjelima, ugovorima i evidencijom rada.

Ključne riječi: Spring Boot, REST API, PostgreSQL, Hibernate ORM, Spring JDBC, MyBatis, SQL, performanse, komunikacija s bazom podataka, anketa.

Abstract

In this final paper, with a practical example of the Spring Boot API, 3 different ways of communicating with the database are compared: Spring JDBC, Hiberante ORM and MyBatis. The goal is to show and analyze the advantages and disadvantages of each technology so that future projects will be able to determine which technology to use.

The paper also describes all other technologies and tools that were used to achieve the results, such as: PostgreSQL, Java, Spring Boot, OpenAPI. For practical comparison, a human resources application was created through which workers, departments, contracts, and work records can be managed.

Keywords: • Spring Boot, REST API, PostgreSQL, Hibernate ORM, Spring JDBC, MyBatis, SQL, performance, database communication, survey.

Grana: 2.09.12 programsko inženjerstvo

ZAVRŠNI ZADATAK br. 0246111279-10-2024-1.

Pristupnik: **Luka Šarić (0246111279)**
Studij: Stručni studij računarstva - Redovan
Smjer: Programsko inženjerstvo - Redovan

Zadatak: **Izrada Spring Boot API-a primjenom različitih vrsta komunikacije s bazom podataka**

Opis zadatka:

1. Opisati korištene tehnologije/alate
2. Opisati komponente API-ja i njihove funkcionalnosti
3. Razviti Spring Boot API u Java programskom jeziku
4. Razviti 3 načina komunikacije s bazom podataka pomoću:
 - a. Hibernate ORM
 - b. MyBatis persistence okvir
 - c. Spring JDBC
5. Napraviti komparacijski pregled načina komuniciranja s bazom podataka.
6. Za pohranjivanje podataka koristiti PostgreSQL bazu podataka.
7. Napraviti javno istraživanje za usporedbu načina komunikacija.

Zadatak uručen pristupniku: 4.4.2025.

Rok za predaju rada: 12.9.2025.

Mentor:

Predsjednik povjerenstva za
završni rad:

Jakob Gračanin, univ. mag. ing. comp., v. pred.

Vedran Tadić, mag.ing.techn.inf., pred.

Sadržaj

1. UVOD.....	1
2. TEHNOLOGIJE.....	2
2.1. POSTGRESQL.....	2
2.2. JAVA.....	3
2.2.1. <i>Project Lombok</i>	4
2.3. SPRING BOOT.....	4
2.3.1. <i>Spring MVC</i>	5
2.3.2. <i>Spring Boot Developer tools</i>	6
2.4. OPENAPI SPECIFIKACIJA.....	7
2.4.1. <i>Swagger</i>	7
2.5. NAČINI KOMUNIKACIJE S BAZOM.....	8
2.5.1. <i>Spring JDBC</i>	9
2.5.2. <i>Hibernate</i>	9
2.5.3. <i>Mybatis</i>	10
3. APLIKACIJA I MODEL.....	11
4. USPOREDBA TEHNOLOGIJA.....	12
4.1. ZAHTEVI DODAVANJA U PROJEKT I OSNOVNA POSTAVA.....	12
4.1.1. <i>Spring JDBC</i>	12
4.1.2. <i>Hibernate</i>	13
4.1.3. <i>MyBatis</i>	13
4.2. IZRADA JEDNOSTAVNIH UPITA.....	13
4.2.1. <i>Spring JDBC</i>	13
4.2.2. <i>Hibernate</i>	14
4.2.3. <i>MyBatis</i>	15
4.3. UPITI KOMPLEKSNIH FILTERA.....	17
4.3.1. <i>Spring JDBC</i>	17
4.3.2. <i>Hibernate</i>	19
4.3.3. <i>MyBatis</i>	21
4.4. TRANSAKCIJE.....	22
4.5. UPITI S UGNJEŽĐENIM PODACIMA.....	22
4.5.1. <i>Spring JDBC</i>	22
4.5.2. <i>Hibernate</i>	23
4.5.3. <i>MyBatis</i>	24
4.6. SKUPNI UNOS.....	25

4.6.1.	<i>Spring JDBC</i>	25
4.6.2.	<i>Hibernate</i>	25
4.6.3.	<i>MyBatis</i>	26
4.7.	DODATNO O TEHNOLOGIJAMA	28
4.7.1.	<i>Dodatni načini kreiranja upita</i>	28
4.7.2.	<i>Imena varijabli u entitetu</i>	28
4.7.3.	<i>Hibernate generirani SQL za kompleksan upit</i>	29
4.8.	PERFORMANSNO TESTIRANJE	30
4.9.	USPOREDNA ANALIZA NAKON IMPLEMENTACIJE	31
5.	ISTRAŽIVANJE I ANKETA	33
6.	ZAKLJUČAK	36
7.	LITERATURA	37
8.	PRILOZI	40

Popis kratica i oznaka

API - Application Programming Interface

REST - Representational State Transfer

ORM - Object-Relational Mapping

JDBC - Java Database Connectivity

ORDBMS - Object-Relational Database Management System

MVCC - Multi-Version Concurrency Control

SQL - Structured Query Language

MVC - Model-View-Controller

AOP - Aspect-Oriented Programming

CRUD - Create, Read, Update, Delete

JDBC - Java Database Connectivity

XML - eXtensible Markup Language

JPQL - Java Persistence Query Language

HQL - Hibernate Query Language

JSON - JavaScript Object Notation

Popis slika

SLIKA 1. PRIKAZ DBEAVER GRAFIČKOG SUČELJA I SQL UREDNIKA. (IZVOR: AUTOR)	3
SLIKA 2. PRIKAZ VRHA TABLICE POPULARNOSTI PROGRAMSKIH JEZIKA (IZVOR: SLIKA ZASLONA [6])	4
SLIKA 3. PRIKAZ SWAGGER UI ZA ENDPOINTE APLIKACIJE. (IZVOR: AUTOR)	8
SLIKA 4. MODEL BAZE PODATKA APLIKACIJE.....	12

Popis kodova

KOD 1. DATOTEKA POSTAVKA SPRING BOOT APLIKACIJE (IZVOR: AUTOR).....	5
KOD 2. PRIMJER KODA SPRING BOOT REST API (IZVOR: AUTOR)	6
KOD 3. PRIMJER OPISIVANJA ENDPOINTA KROZ ANOTACIJE.....	7
KOD 4. PRIMJER DAVANJA OPISA I PRIMJERA ULAZU/IZLAZU	7
KOD 5. PRIMJER JEDNOSTAVNOG UPITA POMOĆU SPRING JDBC	9
KOD 6. PRIMJER KODA JEDNOSTAVNOG UPITA POMOĆU HIBERNATE ORM	9
KOD 7. PRIMJER KORIŠTENJA PREDEFINIRANE METODE U HIBERNATE ORM.....	10
KOD 8. PRIMJER XML-A MAPPERA ZA MYBATIS	10
KOD 9. PRIMJER KODA MAPPERA ZA MYBATIS	10
KOD 10. KOD ZA LOGIRANJE SPRING JDBC UPITA	12
KOD 11. PRIMJER POSTAVKA ZA HIBERNATE	13
KOD 12. PRIMJER POSTAVKA ZA MYBATIS	13
KOD 13. PRIMJER ENTITETA JEDNOSTAVNOG UPITA.....	14
KOD 14. PRIMJER KORIŠTENJA MYBATIS ANOTACIJE U SUČELJU	16
KOD 15. PRIMJER IMPLEMENTACIJE KOMPLEKSNOG FILTERA SPRING JDBC.....	18
KOD 16. PRIMJER KOMPLEKSNOG UPITA HIBERNATE	20
KOD 17. PRIMJER KOMPLEKSNOG UPITA MYBATIS	22
KOD 18. PRIMJER UGNIJEŽĐENOG PODATKA U JSON FORMATU.....	22
KOD 19. PRIMJER KREIRANJA UGNIJEŽĐENOG OBJEKTA SPRING JDBC.....	23
KOD 20. KOD ENTITETA UGNIJEŽĐENOG OBJEKTA HIBERNATE	23
KOD 21. KOD REPOZITORIJA UGNIJEŽĐENOG OBJEKTA HIBERNATE	24
KOD 22. XML KOD UGNIJEŽĐENOG OBJEKTA MYBATIS.....	24
KOD 23. PRIMJER SKUPNOG UNOSA S SPRING JDBC	25
KOD 24. PRIMJER POSTAVKA ZA SKUPNI UNOS HIBERNATE	26
KOD 25. PRIMJER KODA ZA SKUPNI UNOS HIBERNATE	26
KOD 26. PRIMJER KODA XML ZA SKUPNI UNOS MYBATIS	27
KOD 27. PRIMJER KODA ZA RUČNO UPRAVLJANJE SKUPNIM UNOSOM MYBATIS	27

KOD 28. PRIMJER ČISTOG SQL-A U HIBERNATEU	28
KOD 29. PRIMJER JPQL UPITA U HIBERNATEU.....	28
KOD 30. PRIMJER KORIŠTENJA @COLUMN	29
KOD 31. SQL GENERIRAN HIBERNATOM	29

Popis grafova

GRAF 1. USPOREDBA TEHNOLOGIJA PREMA METRIKAMA (SVI PODACI)	30
GRAF 2. USPOREDBA TRAJANJA UPITA TEHNOLOGIJA KROZ BROJ UPITA.....	30
GRAF 3. RASPODJELA ANKETE PO ISKUSTVU	33
GRAF 4. PROSJEČNO ZADOVOLJSTVO S TEHNOLOGIJAMA.....	34
GRAF 5. PROSJEČNE OCJENE SVIH KATEGORIJA I PITANJA PO TEHNOLOGIJI	34

Popis tablica

TABLICA 1. UPOREDNA ANALIZA TEHNOLOGIJA	31
---	----

1. Uvod

U suvremenom razvoju web servisa, komunikacija između aplikacije i baze podataka treba biti brza, pouzdana te skalabilna. U ovom završnom radu na temelju Java REST API-ja napravljenog pomoću Spring Boot okvira, testiraju se tri različita načina moderne komunikacije. Uspoređuju se prednosti i mane između Hibernate ORM-a, MyBatis *persistence* okvira te Spring JDBC (Malo veća apstrakcija običnog JDBC).

Pitanje komunikacije između aplikacije i baze podataka uspoređivat će se po vremenu potrebno za razvoj, održivosti, čitljivosti te brzini. Važnost odgovora na ovo pitanje proizlazi zbog potrebe da se pri svakom novom projektu na početku odabere najprikladniji način. Ovaj odabir može biti zahtjevan i nepredvidiv kada se samo teoretizira te iz tog razloga je vrijedno imati praktični primjer.

Na početku ovog rada opisat će se potrebne tehnologije i alati za napraviti i testirati aplikaciju. U drugom poglavlju detaljno će se opisati API i njegove funkcionalnosti. Treće poglavlje će prikazati model baze. Četvrto poglavlje će se sadržavati od prikaza implementacije API-ja za svaki navedeni način komunikacije i njihove usporedbe. U petom poglavlju analizirat će se istraživanje provedeno anketom iz prakse, te na posljétku zaključak i preporuke.

2. Tehnologije

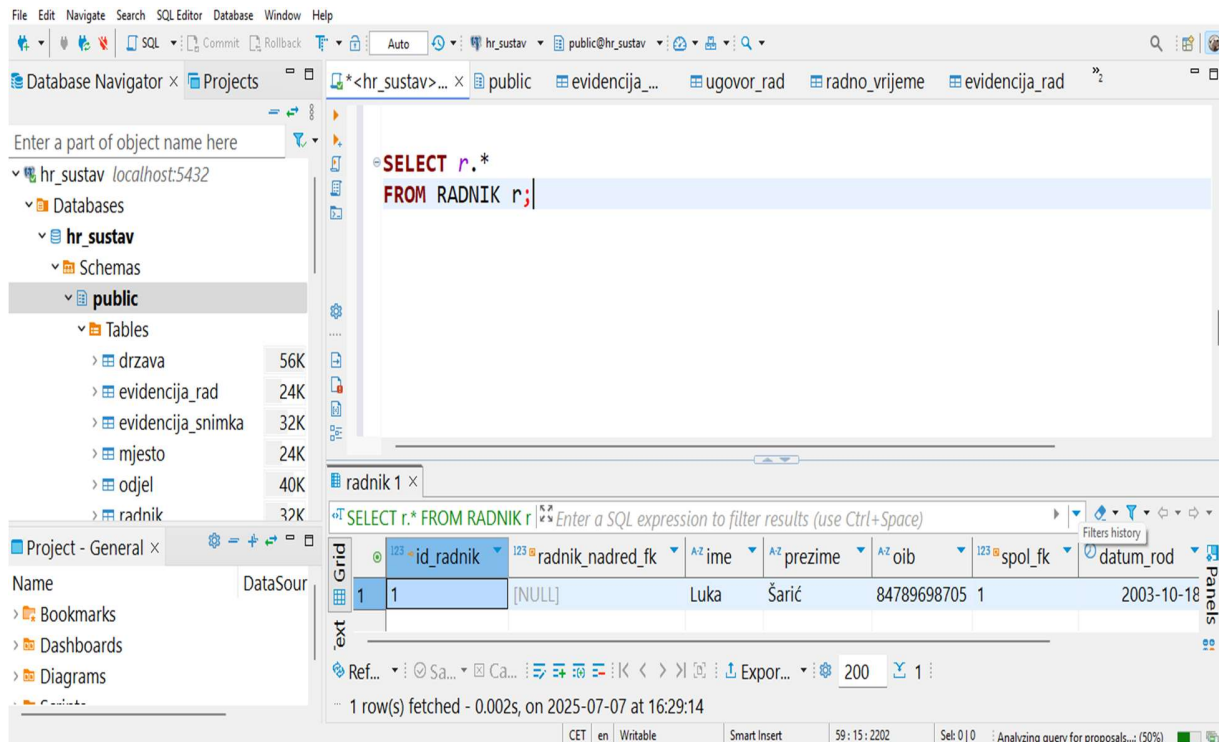
2.1.PostgreSQL

PostgreSQL je objektno-relacijski sustav za upravljanje relacijskim bazama (eng. Object-Relational Database Management System – ORDBMS), koji je nastao na temeljima sustava POSTGRES verzije 4.2. Razvijen je na Sveučilištu Kalifornije u Berkeleyju. Riječ je o projektu otvorenog koda koji je donio nove koncepte koje su komercijalni sustavi baza podataka prihvatili znatno kasnije. Ističe se svojom snažnom podrškom SQL standarda dok još uvijek podržava mnoge moderne značajke, kao što su kompleksni upiti, strani ključevi, okidači (eng. triggers), ažurirajući pogledi, transakcijski integritet i viševerzijska kontrola konkurentnosti (MVCC). Također, zbog svoje otvorenosti, PostgreSQL se može proširiti korisničkim definiranim tipovima podataka, funkcijama, operatorima, agregatnim funkcijama, metodama indeksiranja te novim proceduralnim programskim jezicima. Zahvaljujući svojoj licenci, PostgreSQL se može slobodno koristiti, mijenjati i distribuirati u privatne, komercijalne i akademske svrhe bez ograničenja [1]

U ovom radu PostgreSQL koristi se kao moderna, besplatna te brza lokalna baza podataka na kojoj se uspoređuju tri različita pristupa podacima u Spring Boot aplikaciji: Spring JDBC, Hibernate i MyBatis. [2]

Za jednostavno modeliranje, dizajniranje te upravljanje bazom podataka koristi se besplatna verzija alata DBeaver. Alat putem grafičkog sučelja omogućuje izmjenu podataka, kreiranje analitičkih izvještaja, te izvoz podataka u različite formate. Također nudi i opcije kao što su SQL urednik s automatskim dovršavanjem, alate za migraciju te još mnogo toga. [3]

Na slici 1. prikazano je grafičko sučelje DBeaver alata s otvorenom konekcijom na bazu, strukturom baze te SQL urednikom.



Slika 1. Prikaz DBeaver grafičkog sučelja i SQL urednika. (izvor: autor)

2.2. Java

Java je objektno orijentirani programski jezik temeljen na klasama koji je istodoban, strogo tipiziran i koristi *garbage collector*. Obično se kompajlira u skup bajtkoda i binarni format kako bi se pokrenuo na Java virtualnom stroju. Java virtualan stroj omogućuje Javi da se kod napiše jednom i pokrene bilo gdje. [4] [5]

U ovom radu Java je odabrana kao programski jezik zbog njegove popularnosti u svijetu i Hrvatskoj. Izabran programski jezik također diktira koje načine komunikacije s bazom će se koristiti. [6]

Na slici 2 prikazana je tablica s 10 najpopularnijih programskih jezika na svijetu.

Jul 2025	Jul 2024	Change	Programming Language		Ratings	Change
1	1			Python	26.98%	+10.85%
2	2			C++	9.80%	-0.53%
3	3			C	9.65%	+0.16%
4	4			Java	8.76%	+0.17%
5	5			C#	4.87%	-1.85%
6	6			JavaScript	3.36%	-0.43%
7	7			Go	2.04%	-0.14%
8	8			Visual Basic	1.94%	-0.13%
9	24	⬆		Ada	1.77%	+0.99%
10	11	⬆		Delphi/Object Pascal	1.77%	-0.12%

Slika 2. Prikaz vrha tablice popularnosti programskih jezika (Izvor: slika zaslona [6])

2.2.1. Project Lombok

Project Lombok je biblioteka napravljena za Javu kako bi smanjila standardan kod i ubrzala proces pisanja koda. Pomoću anotacija ima mogućnost kreirati *gettere*, *settere*, *konstrukture*, *buildere* i još mnogo toga. [7] Neke funkcionalnosti kao var koji služi kako ne bi trebali eksplicitno definirati tip varijable su bile dodane u Lombok prije nego u Javu. [8] [9]

2.3.Spring Boot

Spring Boot je prema svojoj licenci okvir otvorenog koda razvijen kao dodatak za postojeći Spring Framework, a njegov cilj je pojednostaviti i ubrzati razvoj Java aplikacija spremnih za produkciju. Za razliku od Spring Frameworka, koji zahtijeva detaljnu konfiguraciju, Spring Boot se temelji na načelu „konvencije umjesto konfiguracije“ (eng. convention over configuration). Ključne značajke Spring Boota su njegova automatska konfiguracija, unaprijed definirani starter paketi koji omogućuju brzu integraciju različitih tehnologija kao što su Spring Data, Spring Security, Spring Data, Spring MVC i mnoge druge. Također dolazi s ugrađenim poslužiteljima

aplikacija poput Tomcata, čime se dodatno pojednostavljuje i ubrzava pokretanje, testiranje i implementaciju na poslužitelju. [10] [11] [12]

U ovom radu, koristi se kako bi se jednostavno napravio robusan API usporediv s pravim komercijalnim. U isječku koda 1. prikazan je primjer postavljenih postavka Spring Boot aplikacije koja koristi profile i varijable sistemske okoline.

```
1. spring.application.name=hrsustav
2. spring.profiles.active=hibernate
3. # spring
4. spring.output.ansi.enabled=ALWAYS
5. # swagger
6. springdoc.swagger-ui.enabled=true
7. # db
8. spring.datasource.url=jdbc:postgresql://localhost:5432/hr_sustav
9. spring.datasource.driverClassName=org.postgresql.Driver
10. spring.datasource.username=${HR_USERNAME}
11. spring.datasource.password=${HR_PASSWORD}
12. spring.sql.init.platform=postgresql
13. # hibernate
14. spring.jpa.hibernate.ddl-auto=none
```

Kod 1. datoteka postavka Spring Boot aplikacije (izvor: autor)

Spring se bazira na tri osnovna koncepta koji pojednostavljaju programiranje i brinu se za održljivost te modularnost koda aplikacije. Prvi je Spring container odnosno ApplicationContext koji kontrolira i brine se o svim *beanovima* aplikacije. *Beanovi* su klase koje se anotiraju s određenim Spring anotacijama te se tako delegiraju ApplicationContextu. [13]

Drugi je AOP (Aspect Oriented Programming) u Springu. On povećava modularnost i daje mogućnost „spajanja“ na određene metode kako bi se izvršio zadani kod prije i/ili nakon metode. Često se koristi za *logiranje*, transakcije i sigurnost. [14]

Zadnji koncept je *Dependency injection* koji povećava čistoću i poboljšava održavanje koda. Tako da se na početku svake klase jasno definira o kojim klasama ta klasa ovisi. Odnosno koje klase će koristiti kako bi izvršila svoj zadatak. [15]

2.3.1. Spring MVC

Detaljnije, aplikacija koristi Spring MVC (Model-View-Controller), komponentu iz Spring Frameworka punim imenom Spring Web MVC. Često se koriste Controller, klasa koja poziva poslovnu logiku odnosno klasu Service, a Service poziva Repository klasu iz koje dobiva resurse potrebne za rad. [16]

Pomoću njega jednostavno je kreirati mrežne aplikacije odnosno aplikacije bazirane na REST (Representational State Transfer) arhitektonskom stilu softvera. Ukratko REST su aplikacije koje komuniciraju bez stanja, imaju uniformno sučelje te mogućnost skalabilnosti zbog URI-ja koji pokazuju na resurse. [17]

U isječku koda 2 može se vidjeti primjer implementacije CRUD operacija za entitet Radnik. Koristi anotaciju `@RestController` za delegaciju Spring containeru, `@AllArgsConstructor` za skraćivanje koda te ostale anotacije kako bi se olakšalo programiranje.

```
1. @RestController
2. @RequestMapping("/api/radnik")
3. @AllArgsConstructor
4. public class RadnikController {
5.     private final RadnikService radnikService;
6.     @PostMapping
7.     public ResponseEntity<Void> createRadnik(@RequestBody RadnikRequest radnik) {
8.         radnik.setIdRadnik(null);
9.         radnikService.createRadnik(radnik);
10.        return ResponseEntity.noContent().build();
11.    }
12.    @GetMapping
13.    public ResponseEntity<List<RadnikDto>> getAllRadnik(RadnikGetRequest request) {
14.        return ResponseEntity.ok().body(radnikService.getAllRadnik(request));
15.    }
16.    @PutMapping
17.    public ResponseEntity<Void> updateRadnik(@RequestBody RadnikRequest radnik) {
18.        radnikService.updateRadnik(radnik);
19.        return ResponseEntity.noContent().build();
20.    }
21.    @DeleteMapping
22.    public ResponseEntity<Void> deleteRadnik(@RequestParam Integer id) {
23.        radnikService.deleteRadnik(id);
24.        return ResponseEntity.noContent().build();
25.    }
26. }
```

Kod 2. Primjer koda Spring Boot REST API (izvor: autor)

2.3.2. Spring Boot Developer tools

Spring Boot Developer tools je modul koji dodatno olakšava razvoj Spring aplikacija kroz brže pokretanje i dodatne opcije *debugginga*. Kako bi ga dodali u projekt dodajemo spring-boot-devtools ovisnosti. U ovom radu koristi se kako bi kada se napravi promjena, brže nego inače pokrenula aplikacija s željenom promjenom. [18]

2.4.OpenAPI specifikacija

OpenAPI specifikacija je format koji opisuje REST API-je. Zapisuje postojeće *endpointe*, ulazne i izlazne parametre, autentifikacijske metode i ostale informacije. Može se pisati u JSON ili YAML formatu. [19]

2.4.1. Swagger

Swagger je skup aplikacija otvorenog koda koje koriste OpenAPI specifikaciju kako bi pomogle dokumentirati, testirati i razviti REST API-je.

U Spring aplikaciju se može dodati ovisnosti `springdoc-openapi-starter-webmvc-ui` koja automatski radi dijelove dokumentacije te omogućuje dodati ostatak kroz određene anotacije.

U isječcima koda 3 i 4, prikazan je način kako se pomoću Swagger anotacija mogu opisati *endpointi*, upiti i odgovori aplikacije.

```
1. @Operation(  
2.     summary = "Dohvati sve odjele",  
3.     description = "Vraća popis svih odjela iz baze podataka.")  
4. @ApiResponse(  
5.     responseCode = "200",  
6.     description = "Popis odjela uspješno dohvaćen")  
7. @GetMapping  
8. public ResponseEntity<List<Odjel>> getAllOdjel() {  
9.     return ResponseEntity.ok().body(odjelService.getAllOdjel());  
10. }
```

Kod 3. Primjer opisivanja endpointa kroz anotacije.

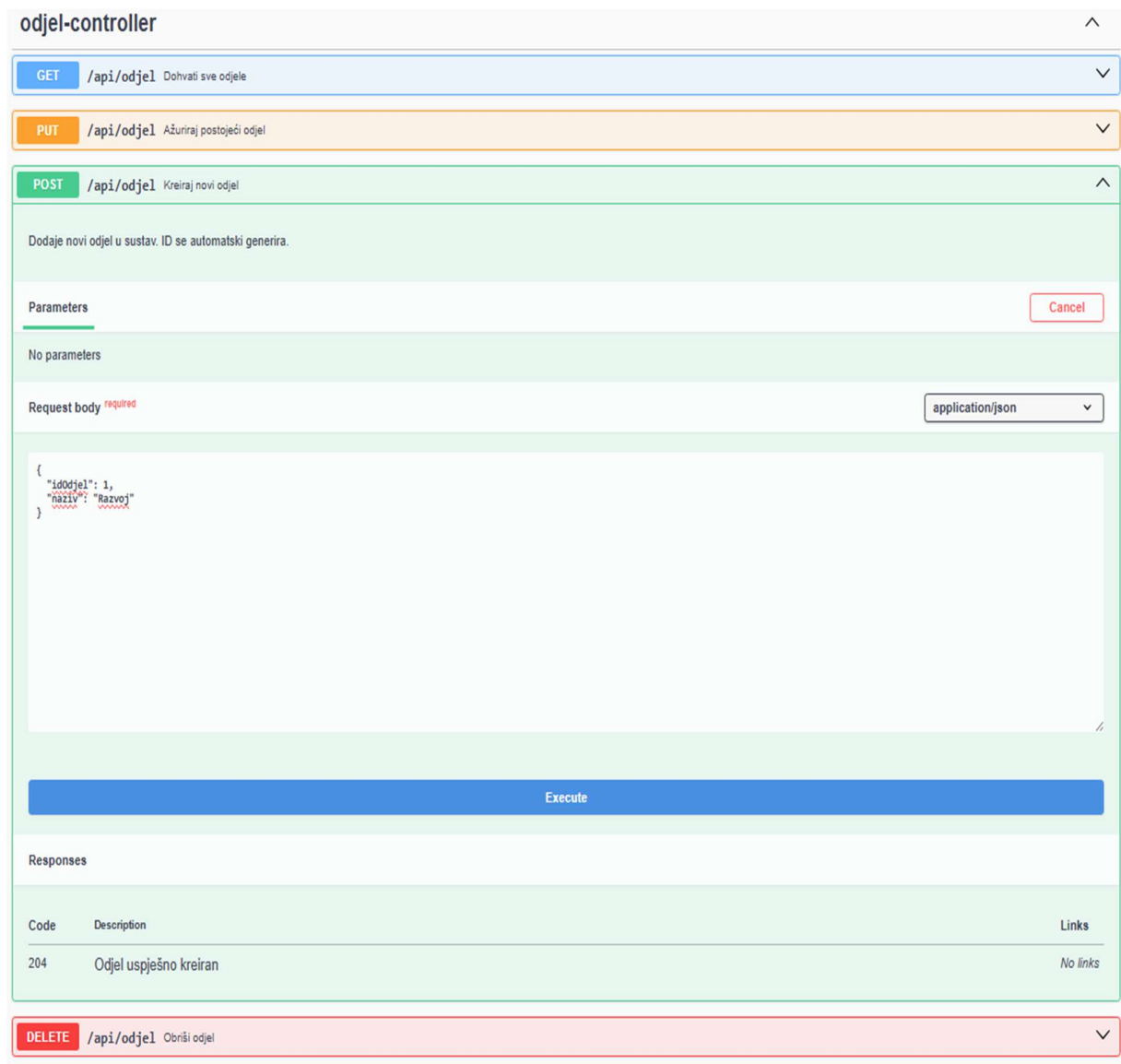
```
1. public class Odjel {  
2.     @Schema(description = "Jedinstveni identifikator odjela", example = "1")  
3.     private Integer idOdjel;  
4.     @Schema(description = "Naziv odjela", example = "Razvoj")  
5.     private String naziv;  
6. }
```

Kod 4. Primjer davanja opisa i primjera ulazu/izlazu

Također se dobiva funkcionalnost pozivanja definiranih *endpointa* te se može koristiti kao prvi sloj testiranja API-ja.

Upravo zbog lakšeg snalaženja u aplikaciji i testiranja aplikacije se koristi u ovom projektu.

Na slici 4. prikazan je primjer izgleda Swagger OpenAPI dokumentaciju upravo za prošle isječke koda. Može se vidjeti kako je primjer već opisan i spreman za testiranje.



Slika 3. Prikaz Swagger UI za endpointe aplikacije. (izvor: autor)

2.5. Načini komunikacije s bazom

Sve tri tehnologije koriste JDBC, i svaka radi apstrakciju do neke razine. U bilo kojem trenutku moguće je pristupiti JDBC.

2.5.1. Spring JDBC

Spring JDBC je nivo apstrakcije koji olakšava upite na bazu podataka ali još uvijek daje mogućnost velike kontrole. Ima klase koje dodatno skraćuju pisanje SQL upita te ima mogućnost spustiti se i na nižu razinu apstrakcije kako bi se detaljnije mogli kontrolirati upiti. [20]

U isječku koda 5. prikazan je primjer koji koristi `@Repository`, `JdbcTemplate` i `BeanPropertyRowMapper` kako bi se napravio jednostavan upit na bazu podataka s zadanim SQL kodom.

```
1. @Repository
2. @AllArgsConstructor
3. public class RadnoMjestoRepositoryR {
4.     private final JdbcTemplate jdbcTemplate;
5.     public List<RadnoMjesto> getAllRadnoMjesto() {
6.         var sql =
7.             """
8.             SELECT
9.                 id_radno_mjesto,
10.                 naziv
11.             FROM radno_mjesto
12.             """;
13.         return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(RadnoMjesto.class));
14.     }
15. }
```

Kod 5. Primjer jednostavnog upita pomoću Spring JDBC

2.5.2. Hibernate

ORM-ov cilj je napraviti potpunu apstrakciju SQL upita. Hibernate ORM to postiže na način da se u klasama entiteta opiše struktura postojeće ili baze podataka koja se želi kreirati. Potom se pomoću predefiniрани metoda prirodnim jezikom mogu napraviti upiti na bazu. Hibernate pomoću definirane strukture može napraviti bazu podataka ili samo upite za mnoge baze podataka i to bez pisanja ikakvog SQL koda. [21]

U isječcima koda 6 i 7 prikazani su primjeri koliko lagano je napraviti jednostavan upit pomoću Hibernate ORM-a. Koristi se mogućnost nasljeđivanja Objektno orijentiranih programskih jezika.

```
1. public interface RadnoMjestoRepositoryH extends JpaRepository<RadnoMjesto, Integer> {}
```

Kod 6. Primjer koda jednostavnog upita pomoću Hibernate ORM

```
1. return radnoMjestoRepository.findAll();
```

Kod 7. Primjer korištenja predefinicirane metode u Hibernate ORM

2.5.3. Mybatis

Mybatis je *persistence* programski okvir za Javu, odnosno biblioteka za spremanje i dohvat podataka iz baze. Mybatis ima razinu apstrakcije ali također daje vrlo jednostavan način za veću kontrolu. Osim olakšavanja pisanja upita, Mybatis olakšava i mapiranje rezultata u objekte. [22] U isječcima koda 8 i 9 prikazani su primjer koda kako napisati i definirati jednostavan upit pomoću Mybatis persistence programskog okvira. SQL se piše u XML datoteke dok se metoda definira u klasi pomoći `@Mapper` anotacije.

```
1. <mapper namespace="hr.tvz.hrsustav.mybatis.mapper.RadnoMjestoMapperB">
2.   <select id="getAllRadnoMjesto" resultType="hr.tvz.hrsustav.entity.RadnoMjesto">
3.     SELECT
4.       id_radno_mjesto,
5.       naziv
6.     FROM radno_mjesto
7.   </select>
8. </mapper>
```

Kod 8. Primjer XML-a mappera za Mybatis

```
1. @Mapper
2. public interface RadnoMjestoMapperB {
3.   public List<RadnoMjesto> getAllRadnoMjesto();
4. }
```

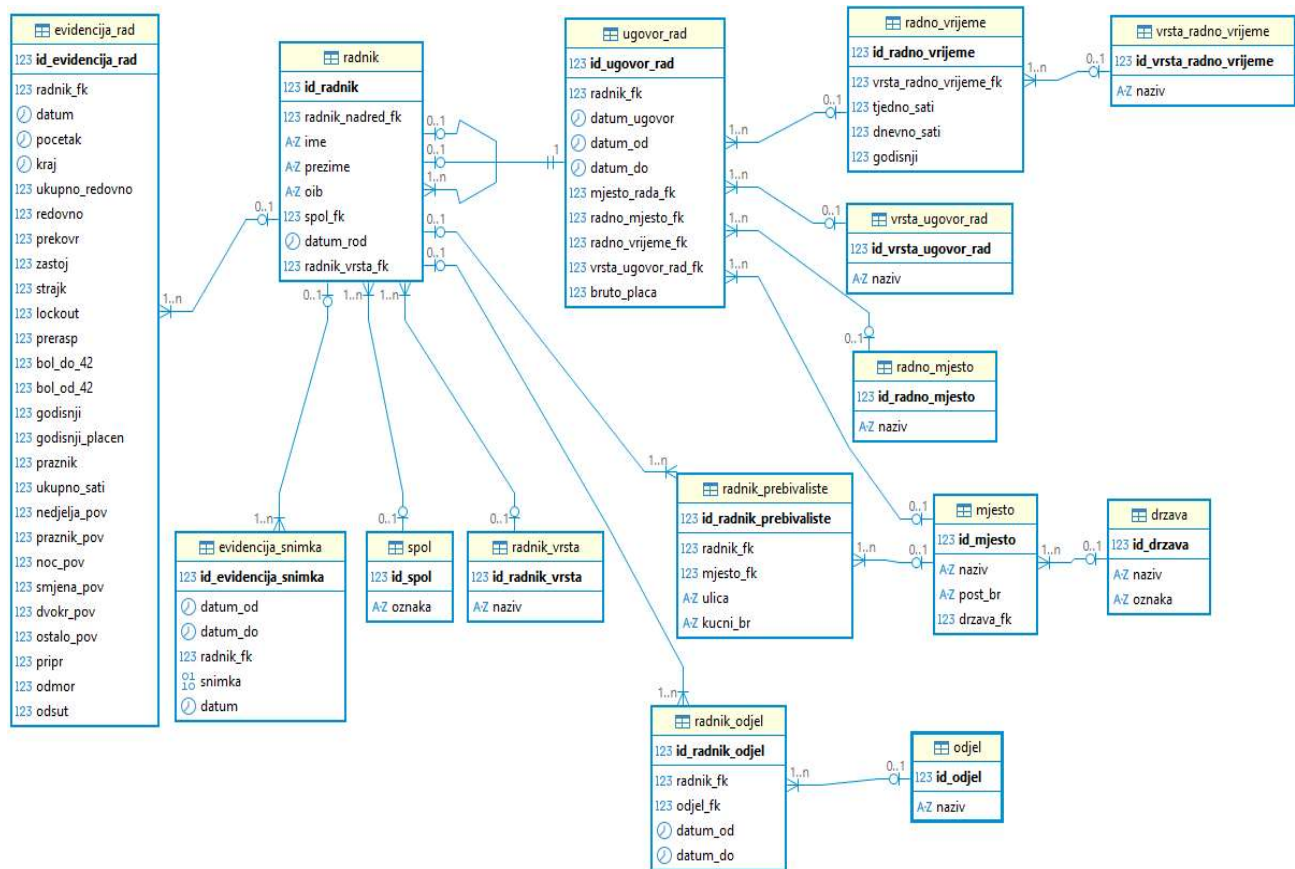
Kod 9. Primjer koda mappera za Mybatis

3. Aplikacija i model

Aplikacija je Spring Boot API. Cilj aplikacije je upravljanje zaposlenicima, podacima zaposlenika, na kojem odjelu rade, njihovim ugovorima i evidencijom sati.

Za potrebe lakog testiranja i pisanja koda te postizanja realističnosti, napravljena je struktura projekta Controller > Service > Query > Repository / Mapper. U ovoj strukturi poziva se metoda Controllera, koja zatim poziva metodu Servicea koja sadrži poslovnu logiku i poziv metode Query sučelja. Ova metoda je zapravo „*wrapper*“ za Repository/Mapper klase odnosno sučelja, koja uz pomoć Spring profila omogućuje da aplikacija brzo i pregledno može mijenjati koju tehnologiju. Repository i Mapper su implementacije tehnologija za komunikaciju s bazom podataka.

Spring profili se koriste da postoji jedan za svaku tehnologiju, što odvaja postavke tehnologija iz .properties. Svaka tehnologija ima svoju implementaciju komunikacije s bazom ali sve koriste istu logiku aplikacije. Model baze podataka uslikan je preko DBEavera i nalazi se na slici 4.



Slika 4. Model baze podatka aplikacije

4. Usporedba tehnologija

4.1. Zahtjevi dodavanja u projekt i osnovna postava

Sve tri tehnologije koriste iste postavke za spajanje na bazu podataka iz koda 1.

4.1.1. Spring JDBC

Kako bi se koristio Spring JDBC unutar Spring Boot aplikacije potrebno je dodati ovisnost „spring-boot-starter-jdbc“. Nije potrebna dodatna postava za korištenje. Opcionalno se može dodati logiranje upita kojih šalje Spring JDBC tako da se u .properties datoteku doda sljedeći kod 10.

```
1. logging.level.org.springframework.jdbc.core=DEBUG
```

Kod 10. Kod za logiranje Spring JDBC upita

4.1.2. Hibernate

Za korištenje Hibernate ORM-a u Spring Boot aplikaciji potrebno je dodati ovisnost „spring-boot-starter-data-jpa“. Za rad nije potrebna dodatno postavljanje, ali je poželjno. U kodu 11. se nalazi properties datoteka s poželjnim kodom i objašnjenjima.

```
1. # Postavka za omogućavanje usporredbe entiteta iz aplikacije s bazom podataka. Može i
   napraviti bazu s danim entitetima.
2. spring.jpa.hibernate.ddl-auto=none
3. # Eksplicitno postavljanje dijalekta inače automatski
4. spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
5. # Logiranje upita te formatiranje tog logiranja
6. spring.jpa.show-sql=true
7. spring.jpa.properties.hibernate.format_sql=true
```

Kod 11. Primjer postavka za Hibernate

4.1.3. MyBatis

Za korištenje MyBatis persistence okvira u Spring Boot aplikaciji potrebna je ovisnost „mybatis-spring-boot-starter“. Nije potrebno dodatno postavljanje za sam rad, ali potrebne su dodatne postavke ako su potrebne određene funkcionalnosti. U kodu 12. nalazi se primjer postavka koji se koristi u aplikaciji ovog rada.

```
1. # po defaultu src/main/resources/mappers/*.xml koristi se ako se koriste .xml za SQL umjesto U
   klasma
2. mybatis.mapper-locations=classpath:mybatis/mapper/**/*.xml
3. # postavka za automatsko mapiranje rezultata upita u klase
4. mybatis.configuration.map-underscore-to-camel-case=true
5. # logiranje konekcije i upita
6. mybatis.configuration.log-impl=org.apache.ibatis.logging.slf4j.Slf4jImpl
7. logging.level.hr.tvz.hrsustav.mybatis.mapper=DEBUG
8. logging.level.org.mybatis=DEBUG
```

Kod 12. Primjer postavka za MyBatis

4.2. Izrada jednostavnih upita

4.2.1. Spring JDBC

Pri izradi jednostavnih upita pomoću Spring JDBC potreban je samo kod 5. koji je već prije bio naveden.

```

1. @Repository
2. @AllArgsConstructor
3. public class RadnoMjestoRepositoryR {
4.     private final JdbcTemplate jdbcTemplate;
5.     public List<RadnoMjesto> getAllRadnoMjesto() {
6.         var sql =
7.             """
8.             SELECT
9.                 id_radno_mjesto,
10.                naziv
11.            FROM radno_mjesto
12.            """;
13.     return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(RadnoMjesto.class));
14.     }
15. }

```

Kod 5. Primjer jednostavnog upita pomoću Spring JDBC

- Anotacija `@Repository` označava da je riječ o klasi zaduženoj za pristup bazi podataka, što Spring Bootu omogućuje automatsko upravljanje tim komponentama.
- `JdbcTemplate` je klasa Springa koja djelomično radi apstrakciju Java JDBC kako bi olakšala upite na bazu podataka.
- `BeanPropertyRowMapper` koristi refleksiju za automatsko mapiranje redaka iz baze podataka na Java objekte, pod uvjetom da imena stupaca odgovaraju nazivima atributa u klasi.
- Uz te alate potrebno je još ručno napisati SQL kod koji odgovara postojećoj bazi podataka.

4.2.2. Hibernate

Za izradu jednostavnog upita u Hibernate ORM-u potrebno je napraviti sučelje navedeno u kodu 6. koji je već prije bio naveden te entitet s specifičnim anotacijama u kodu 14.

```

1. @Entity
2. public class RadnoMjesto {
3.     @Id
4.     @GeneratedValue(strategy = GenerationType.IDENTITY)
5.     private Integer idRadnoMjesto;
6.     private String naziv;
7. }

```

Kod 13. Primjer entiteta jednostavnog upita

```

1. public interface RadnoMjestoRepositoryH extends JpaRepository<RadnoMjesto, Integer> {}

```

Kod 6. Primjer koda jednostavnog upita pomoću Hibernate ORM

- Anotacija `@Entity` označava da je klasa JPA entitet, čime omogućuje Spring Bootu i Hibernateu da mapiraju tu klasu na relacijsku tablicu u bazi podataka.
- Anotacija `@Id` označava atribut koji predstavlja primarni ključ u bazi podataka.
- Anotacija `@GeneratedValue` definira strategiju generiranja vrijednosti primarnog ključa, tj. omogućuje automatsko generiranje identifikatora pri unosu novih zapisa u bazu.
- `JpaRepository` je Spring Data sučelje koje omogućuje korištenje ugrađenih CRUD operacija i drugih metoda bez potrebe za pisanjem vlastitih SQL upita. Entitet i primarni ključ se navode kao tipovi u dijamantnom operatoru (`<>`).
- Prva klasa u dijamantnom operatoru je entitet, a druga je klasa primarnog ključa.

Hibernate automatski generira SQL kod, mapiranje i metode.

4.2.3. MyBatis

Za izradu jednostavnog upita pomoću MyBatis persistence okvira potrebno je koristiti prije navedene kodove 8. i 9.

```
1. <mapper namespace="hr.tvz.hrsustav.mybatis.mapper.RadnoMjestoMapperB">
2.   <select id="getAllRadnoMjesto" resultType="hr.tvz.hrsustav.entity.RadnoMjesto">
3.     SELECT
4.       id_radno_mjesto,
5.       naziv
6.     FROM radno_mjesto
7.   </select>
8. </mapper>
```

Kod 8. Primjer XML-a mappera za Mybatis

```
1. @Mapper
2. public interface RadnoMjestoMapperB {
3.   public List<RadnoMjesto> getAllRadnoMjesto();
4. }
```

Kod 9. Primjer koda mappera za Mybatis

- Element `<mapper>` u .xml datoteci sadrži atribut `namespace` koji mora odgovarati potpuno kvalificiranom nazivu sučelja (engl. fully qualified name) koje se koristi za definiranje upita. Unutar tog elementa nalaze se ostali elementi poput `<select>`, `<update>`, `<delete>` i `<insert>`, kojima se definiraju pojedinačni SQL upiti.

- Element <select> koristi se za definiranje SELECT upita u MyBatis .xml mapper datoteci.

Sadrži SQL upit i attribute:

- id: mora odgovarati nazivu metode u sučelju
- resultType: definira tip podatka koji metoda vraća.

- Sučelje se mora anotirati s @Mapper kako bi Spring Boot znao kako ga koristiti

MyBatis podržava i anotacije (npr. @Select) kao alternativu .xml konfiguraciji, no ovaj pristup ograničava fleksibilnost, posebno kod složenijih upita, pa će se u radu se koristiti XML pristup. U kodu 14. nalazi se primjer koji koristi anotaciju @Select. Potrebno je samo upisati SQL kod u anotaciju.

```
1. @Mapper
2. public interface RadnoMjestoMapperB {
3.     @Select("
4.         SELECT
5.             id_radno_mjesto,
6.             naziv
7.         FROM radno_mjesto
8.     ")
9.     List<RadnoMjesto> getAllRadnoMjesto();
10. }
```

Kod 14. Primjer korištenja MyBatis anotacije u sučelju

4.3. Upiti kompleksnih filtera

4.3.1. Spring JDBC

Spring JDBC nema dodatnu potporu za pisanje kompleksnih filtera za upite, ali se može napraviti vlastita implementacija. U kodu 15. nalazi se primjer implementacije.

```
1. public List<RadnikDto> getAllRadnik(RadnikGetRequest request) {
2.     var sql = new StringBuilder("")
3.         SELECT DISTINCT ON (r.id_radnik)
4.             r.id_radnik,
5.             r.radnik_nadred_fk,
6.             r.ime,
7.             r.prezime,
8.             r.oib,
9.             s.oznaka AS spol,
10.            r.datum_rod,
11.            rv.naziv AS radnikVrstaNaziv,
12.            ro.odjel_fk,
13.            ur.id_ugovor_rad,
14.            ur.datum_od AS datumOdUgovorRad,
15.            ro.datum_od AS datumOdRadnikOdjel
16.        FROM radnik r
17.        LEFT JOIN spol s ON s.id_spol = r.spol_fk
18.        LEFT JOIN radnik_vrsta rv ON rv.id_radnik_vrsta = r.radnik_vrsta_fk
19.        LEFT JOIN radnik_odjel ro ON ro.radnik_fk = r.id_radnik
20.        LEFT JOIN ugovor_rad ur ON ur.radnik_fk = r.id_radnik
21.        WHERE 1=1
22.    };
23.    Map<String, Object> params = new HashMap<>();
24.    if (request.getIdRadnik() != null) {
25.        sql.append(" AND r.id_radnik = :idRadnik");
26.        params.put("idRadnik", request.getIdRadnik());
27.    }
28.    if (request.getRadnikNadredFk() != null) {
29.        sql.append(" AND r.radnik_nadred_fk = :radnikNadredFk");
30.        params.put("radnikNadredFk", request.getRadnikNadredFk());
31.    }
32.    if (request.getIme() != null) {
33.        sql.append(" AND r.ime = :ime");
34.        params.put("ime", request.getIme());
35.    }
36.    if (request.getPrezime() != null) {
37.        sql.append(" AND r.prezime = :prezime");
38.        params.put("prezime", request.getPrezime());
39.    }
40.    if (request.getOib() != null) {
41.        sql.append(" AND r.oib = :oib");
42.        params.put("oib", request.getOib());
43.    }
44.    if (request.getSpolFk() != null) {
45.        sql.append(" AND r.spol_fk = :spolFk");
46.        params.put("spolFk", request.getSpolFk());
47.    }
48.    if (request.getDatumRod() != null) {
49.        sql.append(" AND r.datum_rod = :datumRod");
50.        params.put("datumRod", request.getDatumRod());
51.    }
```

```

52.     if (request.getRadnikVrstaFk() != null) {
53.         sql.append(" AND r.radnik_vrsta_fk = :radnikVrstaFk");
54.         params.put("radnikVrstaFk", request.getRadnikVrstaFk());
55.     }
56.     if (Boolean.TRUE.equals(request.getInOdjel())) {
57.         sql.append("""
58.             AND ro.datum_od <= CURRENT_DATE
59.             AND (ro.datum_do >= CURRENT_DATE OR ro.datum_do IS NULL)
60.         """);
61.     }
62.     if (Boolean.TRUE.equals(request.getIsZaposlen())) {
63.         sql.append("""
64.             AND ur.datum_od <= CURRENT_DATE
65.             AND (ur.datum_do IS NULL OR ur.datum_do >= CURRENT_DATE)
66.         """);
67.     }
68.     sql.append("""
69.         ORDER BY
70.             r.id_radnik,
71.             ro.datum_od DESC,
72.             ur.datum_od DESC;
73.     """);
74.     return namedJdbcTemplate.query(sql.toString(), params, new
BeanPropertyRowMapper<>(RadnikDto.class));
75. }

```

Kod 15. Primjer implementacije kompleksnog filtera Spring JDBC

- SQL String je potrebno nadograđivati pomoću StringBuildera (zbog performansi).
- Potrebno je napomenuti da dodavanje parametra u mapu nije potrebno raditi u if blokovima kako je u primjeru, jer SpringBoot ignorira parametar iz mape ako se ne nalazi u SQL Stringu.
- Moguće je koristiti i „var params = new BeanPropertySqlParameterSource(request);“ umjesto ručnog mapiranja parametra. Što će skratiti kod, ali zbog korištenja refleksije će i minimalno povećati trajanje upita.

4.3.2. Hibernate

Hibernate sadrži potporu za kompleksne filtere pomoću JPA Criteria API. Primjer implementacije nalazi se u kodu 16.

```
1. public class RadnikRepositoryC {
2.
3.     private final EntityManager entityManager;
4.
5.     public List<RadnikDto> findAllRadnik(RadnikGetRequest request) {
6.
7.         CriteriaBuilder cb = entityManager.getCriteriaBuilder();
8.         CriteriaQuery<RadnikDto> query = cb.createQuery(RadnikDto.class);
9.         Root<Radnik> r = query.from(Radnik.class);
10.
11.         Join<Radnik, Spol> s = r.join("spolFk", JoinType.LEFT);
12.         Join<Radnik, RadnikVrsta> rv = r.join("radnikVrstaFk", JoinType.LEFT);
13.         Join<Radnik, RadnikOdjel> ro = r.join("radnikOdjel", JoinType.LEFT);
14.         Join<Radnik, UgovorRad> ur = r.join("ugovorRad", JoinType.LEFT);
15.
16.         query.select(cb.construct(
17.             RadnikDto.class,
18.             r.get("idRadnik"),
19.             r.get("radnikNadredFk").get("idRadnik"),
20.             r.get("ime"),
21.             r.get("prezime"),
22.             r.get("oib"),
23.             s.get("oznaka"),
24.             r.get("datumRod"),
25.             rv.get("naziv"),
26.             ro.get("odjelFk").get("idOdjel"),
27.             ur.get("idUgovorRad"),
28.             ro.get("datumOd"),
29.             ur.get("datumOd")));
30.
31.         List<Predicate> predicates = new ArrayList<>();
32.
33.         addPredicateIfNotNull(predicates, cb, r.get("idRadnik"),
34.             request.getIdRadnik());
35.         addPredicateIfNotNull(predicates, cb, r.get("radnikNadredFk"),
36.             request.getRadnikNadredFk());
37.         addPredicateIfNotNull(predicates, cb, r.get("ime"), request.getIme());
38.         addPredicateIfNotNull(predicates, cb, r.get("prezime"),
39.             request.getPrezime());
40.         addPredicateIfNotNull(predicates, cb, r.get("oib"), request.getOib());
41.         addPredicateIfNotNull(predicates, cb, r.get("spolFk"), request.getSpolFk());
42.         addPredicateIfNotNull(predicates, cb, r.get("datumRod"),
43.             request.getDatumRod());
44.         addPredicateIfNotNull(predicates, cb, r.get("radnikVrstaFk"),
45.             request.getRadnikVrstaFk());
46.
47.         if (Boolean.TRUE.equals(request.getInOdjel())) {
48.             predicates.add(cb.lessThanOrEqualTo(ro.get("datumOd"),
49.                 cb.currentDate()));
50.             predicates.add(cb.or(
51.                 cb.greaterThanOrEqualTo(ro.get("datumDo"),
52.                     cb.currentDate()),
53.                 cb.isNull(ro.get("datumDo"))));
54.         }
55.     }
```

```

49.         if (Boolean.TRUE.equals(request.getIsZaposlen())) {
50.             predicates.add(cb.lessThanOrEqualTo(ur.get("datumOd"),
cb.currentDate()));
51.             predicates.add(cb.or(
52.                 cb.greaterThanOrEqualTo(ur.get("datumDo"),
cb.currentDate()),
53.                 cb.isNull(ur.get("datumDo"))));
54.         }
55.
56.         query.where(cb.and(predicates.toArray(new Predicate[0])));
57.         query.distinct(true);
58.
59.         query.orderBy(
60.             cb.asc(r.get("idRadnik")),
61.             cb.desc(ro.get("datumOd")),
62.             cb.desc(ur.get("datumOd")));
63.
64.         return entityManager.createQuery(query).getResultList();
65.     }
66.
67.     private <T> void addPredicateIfNotNull(List<Predicate> predicates, CriteriaBuilder cb,
Path<T> path, T value) {
68.         if (value != null) {
69.             predicates.add(cb.equal(path, value));
70.         }
71.     }
72. }

```

Kod 16. Primjer kompleksnog upita Hibernate

- Kreiranje upita (JPA Criteria API): koristi se CriteriaBuilder i CriteriaQuery<RadnikDto>
- query.select(cb.construct(RadnikDto.class, ...))
- Criteria posjeduje metode koje odogvaraju ključnim riječima iz SQL-a, kao što su: join, select, where, distinct, orderBy. Pomoću tih metoda potrebno je konstruirati SQL upit.
- Criteria također ima klasu Predicates s kojom je moguće dodati logiku u SQL upit.
- entityManager.createQuery(query).getResultList() vraća listu rezultata.
- Hibernate ne podržava DISTINCT ON ključni riječ, kao što to Spring JDBC i MyBatis podržavaju
- Važno je napomenuti da je metoda addPredicateIfNotNull ručno dodana metoda za skraćivanje koda.

4.3.3. MyBatis

MyBatis sadrži potporu za kompleksne filtere pomoću dodatnih XML elementa. U kodu 17. nalazi se primjer implementacije.

```
1. <select id="getAllRadnik" resultType="hr.tvz.hrsustav.dto.RadnikDto"
2.     parameterType="hr.tvz.hrsustav.request.RadnikGetRequest">
3.     SELECT DISTINCT ON (r.id_radnik)
4.         r.id_radnik,
5.         r.radnik_nadred_fk,
6.         r.ime,
7.         r.prezime,
8.         r.oib,
9.         s.oznaka AS spol,
10.        r.datum_rod,
11.        rv.naziv AS radnikVrstaNaziv,
12.        ro.odjel_fk,
13.        ur.id_ugovor_rad,
14.        ur.datum_od AS datumOdUgovorRad,
15.        ro.datum_od AS datumOdRadnikOdjel
16.    FROM radnik r
17.    LEFT JOIN spol s ON s.id_spol = r.spol_fk
18.    LEFT JOIN radnik_vrsta rv ON rv.id_radnik_vrsta = r.radnik_vrsta_fk
19.    LEFT JOIN radnik_odjel ro ON ro.radnik_fk = r.id_radnik
20.    LEFT JOIN ugovor_rad ur ON ur.radnik_fk = r.id_radnik
21.    <where>
22.        <if test="idRadnik != null">
23.            AND r.id_radnik = #{idRadnik}
24.        </if>
25.        <if test="radnikNadredFk != null">
26.            AND r.radnik_nadred_fk = #{radnikNadredFk}
27.        </if>
28.        <if test="ime != null">
29.            AND r.ime = #{ime}
30.        </if>
31.        <if test="prezime != null">
32.            AND r.prezime = #{prezime}
33.        </if>
34.        <if test="oib != null">
35.            AND r.oib = #{oib}
36.        </if>
37.        <if test="spolFk != null">
38.            AND r.spol_fk = #{spolFk}
39.        </if>
40.        <if test="datumRod != null">
41.            AND r.datum_rod = #{datumRod}
42.        </if>
43.        <if test="radnikVrstaFk != null">
44.            AND r.radnik_vrsta_fk = #{radnikVrstaFk}
45.        </if>
46.        <if test="inOdjel == true">
47.            AND ro.datum_od <![CDATA[<=]]> CURRENT_DATE
48.            AND (datum_do <![CDATA[>=]]> CURRENT_DATE OR datum_do IS NULL)
49.        </if>
50.        <if test="isZaposlen == true">
51.            AND ur.datum_od <![CDATA[<=]]> CURRENT_DATE
52.            AND (ur.datum_do IS NULL
53.                OR ur.datum_do <![CDATA[>=]]> CURRENT_DATE)
54.        </if>
55.    </where>
```

```

54.         ORDER BY
55.             r.id_radnik,
56.             ro.datum_od DESC,
57.             ur.datum_od DESC;
58. </select>

```

Kod 17. Primjer kompleksnog upita MyBatis

- Pomoću Elementa <if> i atributa test, može se lagano dodati logika unutar SQL s kojom se može napraviti kompleksan filter.
- Potrebno je koristiti <![CDATA[]]> oko znakova „<“ i „>“ zbog XML formata.

4.4. Transakcije

Sve tri tehnologije automatski koriste transakcije kod jednostavnih upita. U slučaju korištenja nekih kompleksnih upita ili slanjem dva upita koji su jedna cjelina, sve tri tehnologije podržavaju Springov @Transactional, koji se stavlja iznad metode u servisu kako bi se metode repozitorija izvršile u jednoj transakciji.

4.5. Upiti s ugniježđenim podacima

Primjer podatka u JSON (JavaScript Object Notation) formatu se nalazi na u kodu 18.

```

1. [
2.   {
3.     "idMjesto": 1,
4.     "naziv": "Split",
5.     "postBr": "21000",
6.     "drzavaFk": {
7.       "idDrzava": 1,
8.       "naziv": "Republika Hrvatska",
9.       "oznaka": "RH"
10.    }
11.  }
12. ]

```

Kod 18. Primjer ugniježđenog podatka u JSON formatu

4.5.1. Spring JDBC

Spring JDBC nema implementiranu potporu za izgradnju ugniježđenih objekata, zbog čega ju je potrebno ručno implementirati. U kodu 19. nalazi se primjer implementacije.

```

1. public List<Mjesto> getAllMjesto() {
2.     var sql = ""
3.     SELECT
4.         m.id_mjesto,
5.         m.naziv,
6.         m.post_br,
7.         d.id_drzava,
8.         d.naziv AS drzava_naziv,
9.         d.oznaka AS drzava_oznaka
10.    FROM mjesto m
11.   JOIN drzava d ON m.drzava_fk = d.id_drzava
12.   "";
13.   return jdbcTemplate.query(sql, (rs, rowNum) -> {
14.       var drzava = new Drzava();
15.       drzava.setIdDrzava(rs.getInt("id_drzava"));
16.       drzava.setNaziv(rs.getString("drzava_naziv"));
17.       drzava.setOznaka(rs.getString("drzava_oznaka"));
18.
19.       var mjesto = new Mjesto();
20.       mjesto.setIdMjesto(rs.getInt("id_mjesto"));
21.       mjesto.setNaziv(rs.getString("naziv"));
22.       mjesto.setPostBr(rs.getString("post_br"));
23.
24.       mjesto.setDrzavaFk(drzava);
25.
26.       return mjesto;
27.   });
28. }

```

Kod 19. Primjer kreiranja ugniježđenog objekta Spring JDBC

- Koristeći *overload* metode query u retku 13. koji očekuje RowMapper<T> kao drugi argument. Može se predati lambda koja govori Spring JDBC kako mapirati dobivene podatke iz ravne strukture u ugniježđenu.

4.5.2. Hibernate

Hibernate podržava ugniježđivanje podataka odnosno to mu je uobičajeno ponašanje. U kodovima 20. i 21. nalazi se primjer potrebnog koda za ovaj upit.

```

1. @Entity
2. public class Mjesto {
3.     @Id
4.     @GeneratedValue(strategy = GenerationType.IDENTITY)
5.     private Integer idMjesto;
6.     private String naziv;
7.     private String postBr;
8.     @ManyToOne
9.     @JoinColumn(name = "drzava_fk")
10.    private Drzava drzavaFk;
11. }

```

Kod 20. Kod entiteta ugniježđenog objekta Hibernate

```
1. public interface MjestoRepositoryH extends JpaRepository<Mjesto, Integer> {}
```

Kod 21. Kod repozitorija ugniježđenog objekta Hibernate

- Uz korištenje anotacije `@Entity`, `@ManyToOne` i `@JoinColumn`, Hibernate automatski kreira ugniježđeni objekt.
- Potreban je još samo uobičajeni repozitorij

4.5.3. MyBatis

MyBatis nema podržano automatsko kreiranje ugniježđenih objekta te je zato potrebno mapirati objekt. U kodu 22. nalazi se primjer potrebnog koda.

```
1.      <resultMap id="mjestoWithDrzavaResultMap" type="hr.tvz.hrsustav.entity.Mjesto">
2.          <id property="idMjesto" column="id_mjesto"/>
3.          <result property="naziv" column="naziv"/>
4.          <result property="postBr" column="post_br"/>
5.
6.          <association property="drzavaFk" javaType="hr.tvz.hrsustav.entity.Drzava">
7.              <id property="idDrzava" column="id_drzava"/>
8.              <result property="naziv" column="drzava_naziv"/>
9.              <result property="oznaka" column="drzava_oznaka"/>
10.         </association>
11.     </resultMap>
12.
13.     <select id="getAllMjesto" resultMap="mjestoWithDrzavaResultMap">
14.         SELECT
15.             m.id_mjesto,
16.             m.naziv,
17.             m.post_br,
18.             d.id_drzava,
19.             d.naziv AS drzava_naziv,
20.             d.oznaka AS drzava_oznaka
21.         FROM mjesto m
22.         JOIN drzava d ON m.drzava_fk = d.id_drzava
23.     </select>
```

Kod 22. XML kod ugniježđenog objekta MyBatis

- Element `<resultMap>` definira mapiranje stupaca u polja entiteta Mjesto
- Element `<association>` mapira ugniježđeni objekt Drzava i veže ga na Mjesto.drzavaFk
- Također je potrebno dodati i odgovarajuće mapper sučelje kao i u prošlim MyBatis primjerima

4.6. Skupni unos

4.6.1. Spring JDBC

Spring JDBC ima ugrađenu potporu za lagano transformaciju unosa u skupne unose. Primjer koda kako postići skupni unos nalazi se u kodu 23.

```
1. public void createRadnikPrebivaliste(List<RadnikPrebivaliste> radnikPrebivalisteList) {
2.     var sql = ""
3.         INSERT INTO radnik_prebivaliste (
4.             radnik_fk,
5.             mjesto_fk,
6.             ulica,
7.             kucni_br
8.         ) VALUES (
9.             :radnikFk,
10.            :mjestoFk,
11.            :ulica,
12.            :kucniBr
13.        )
14.     "";
15.     SqlParameterSource[] batchArgs = SqlParameterSourceUtils
16.         .createBatch(radnikPrebivalisteList.toArray());
17.     namedJdbcTemplate.batchUpdate(sql, batchArgs);
18. }
```

Kod 23. Primjer skupnog unosa s Spring JDBC

- Korištenjem `SqlParameterSourceUtils.createBatch`, mapira se lista objekta u SQL parametre.
- Korištenjem `.batchUpdate` Spring JDBC se brine za transakciju i skupno unosi podatke u bazu podataka.

Važno je napomenuti da ova implementacija ne vraća automatski generirane primarne ključeve koja je baza dodijelila redcima. U slučaju da su primarni ključevi potrebni trebat će napraviti vlastitu implementaciju pomoću SQL sekvenci, primarne ključeve generirati u aplikaciji ili u slučaju PostgreSQL baze podataka u SQL dodati „RETURNING id“.

4.6.2. Hibernate

Hibernate podržava skupni unos te koristi optimizatore za sekvence, među kojima je `pooled-lo`. Kod `pooled-lo`, kada se koristi `SEQUENCE` s `allocationSize > 1`, Hibernate jednom uzima vrijednost iz sekvence (`hi`) i zatim u memoriji generira preostalih `allocationSize - 1` ID-eva

(lo), čime se bitno smanjuje broj poziva bazi i omogućuje stvarno JDBC skupno umetanje. Primjer se nalazi u kodu 24. i 25.

```
1. # podjela skupnog unosa
2. spring.jpa.properties.hibernate.jdbc.batch_size=50
3. # grupira sql upite po tablicama
4. spring.jpa.properties.hibernate.order_inserts=true
5. spring.jpa.properties.hibernate.order_updates=true
6. # biranje optimirazora
7. spring.jpa.properties.hibernate.id.optimizer.pooled.preferred=pooled-lo
8. # dodatno logiranje
9. spring.jpa.properties.hibernate.generate_statistics=true
10. logging.level.org.hibernate.stat=DEBUG
```

Kod 24. Primjer postavka za skupni unos Hibernate

```
1. @Entity
2. public class RadnikPrebivaliste {
3.     @Id
4.     @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "radnik_preb_seq")
5.     @SequenceGenerator(
6.         name="radnik_preb_seq",
7.         sequenceName = "radnik_prebivaliste_seq",
8.         allocationSize = 50
9.     )
10.     private Integer idRadnikPrebivaliste;
11.     private Integer radnikFk;
12.     private Integer mjestoFk;
13.     private String ulica;
14.     private String kucniBr;
15. }
```

Kod 25. Primjer koda za skupni unos Hibernate

- Anotacija `@GeneratedValue` služi kako bi se uključilo generiranje primarnih ključeva pomoću sekvence, također definira ime sekvence u bazi podataka.
- Anotacija `@SequenceGenerator(..., allocationSize = 50)` definira veličinu bloka ID-eva koje Hibernate preuzima unaprijed

Važno je napomenuti da je za ovu implementaciju potrebno napraviti sekvencu u bazi koja se uvećava za 50 odnosno za veličinu `allocationSize`. Korištenjem sekvenci moguće je doći do stanja da je druga aplikacija napravila unos podatka koji je zbog tog zauzeo određeni ID u bazi podataka.

4.6.3. MyBatis

MyBatis ne podržava skupni unos podatka, već traži da se on sam implementira. Za ovu aplikaciju bio je implementiran način koji radi slično Hibernate implementaciji, ali preuzima

varijabilnu veličinu bloka ID-eva iz baze podataka. U kodu 26. nalazi se potrebna implementacija u .XML mapperu MyBatisa, dok se u kodu 27. nalazi ručno implementirano rukovanje skupovima.

```
1.      <select id="getNextIds" resultType="java.lang.Integer" parameterType="int">
2.          SELECT nextval('radnik_prebivaliste_seq') FROM generate_series(1, #{count})
3.      </select>
4.      <insert id="createRadnikPrebivaliste" parameterType="list">
5.          INSERT INTO radnik_prebivaliste (
6.              id_radnik_prebivaliste,
7.              radnik_fk,
8.              mjesto_fk,
9.              ulica,
10.             kucni_br
11.         ) VALUES
12.         <foreach collection="list" item="i" separator=",">
13.             (#{i.idRadnikPrebivaliste}, #{i.radnikFk},
14.             #{i.mjestoFk}, #{i.ulica}, #{i.kucniBr})
15.         </foreach>
16.     </insert>
```

Kod 26. Primjer koda XML za skupni unos MyBatis

```
1. @Transactional
2. public void createRadnikPrebivaliste(List<RadnikPrebivaliste> radnikPrebivalisteList) {
3.     final int batchSize = 50;
4.     for (var i = 0; i < radnikPrebivalisteList.size(); i += batchSize) {
5.         var end = Math.min(i + batchSize, radnikPrebivalisteList.size());
6.         var batch = radnikPrebivalisteList.subList(i, end);
7.         var ids = radnikPrebivalisteMapper.getNextIds(batchSize);
8.         for (int j = 0; j < batch.size(); j++) {
9.             batch.get(j).setIdRadnikPrebivaliste(ids.get(j));
10.        }
11.        radnikPrebivalisteMapper.createRadnikPrebivaliste(batch);
12.    }
13. }
```

Kod 27. Primjer koda za ručno upravljanje skupnim unosom Mybatis

- Korištenjem `nextval()` i `generate_series()` aplikacija dobiva listu proizvoljne veličine rezerviranih ID-a
- Korištenjem `foreach` petlje unutar `<insert>` elementa MyBatis piše jednu SQL naredbu za cijeli skupni unos
- Nakon što aplikacija dobije listu ID-a pomoću `getNextIds`, ručno ih postavlja u objekte kako bi kasnije upisala te objekte s ID-om u bazu pomoću `createRadnikPrebivaliste`

4.7. Dodatno o tehnologijama

4.7.1. Dodatni načini kreiranja upita

Hibernate podržava JPQL i čisti SQL, primjeri se nalaze u kodovima 28. i 29.

```
1.     @Modifying
2.     @Query(value = ""
3.         DELETE FROM radno_vrijeme WHERE id_radno_vrijeme = (
4.             SELECT radno_vrijeme_fk
5.             FROM ugovor_rad WHERE id_ugovor_rad = :id
6.         );
7.         "" nativeQuery = true)
8.     int deleteRadnoVrijemeByUgovorRadId(@Param("id") Integer id);
```

Kod 28. Primjer čistog SQL-a u Hibernateu

- Korištenjem anotacije `@Query` i postavljanjem parametra `nativeQuery` na točno, moguće je napisati čisti SQL unutar Hibernate sučelja.

```
1.     @Query("""
2.         SELECT new hr.tvz.hrsustav.dto.RadnikOdjelDto(
3.             r.idRadnikOdjel,
4.             r.radnikFk.idRadnik,
5.             r.odjelFk.naziv,
6.             r.datumOd,
7.             r.datumDo)
8.         FROM RadnikOdjel r
9.         WHERE r.radnikFk.idRadnik = :radnikFk
10.    """)
11.    List<RadnikOdjelDto> findByRadnikFk_IdRadnik(Integer radnikFk);
```

Kod 29. Primjer JPQL upita u Hibernateu

- Korištenjem anotacije `@Query` koristi se JPQL (Java Persistence Query Language) standard koji je sličan SQL-u ali radi nad entitetima, a ne nad tablicama.
- Ima i mogućnosti da se pozove konstruktor klase. Time omogućuje slanje podataka u klasu koja nije početno definiran entitet.
- Hibernate osim JPQL standarda ima i HQL (Hibernate Query Language) što je Hibernate specifična implementacija s još dodatnih funkcionalnosti.

4.7.2. Imena varijabli u entitetu

Za neke krajnje slučajeve ili ako ime varijable u entitetu ne odgovara imenu stupca, moguće je koristiti anotaciju `@Column`. Za rješenje istog problema Spring JDBC i MyBatis moraju koristiti SQL *alias*. Primjer korištenje anotacije nalazi se u kodu 30.

```

1.      @Column(name = "bol_od_42")
2.      private Double bolOd42;

```

Kod 30. Primjer korištenja @Column

Vrijednost dodijeljena name parametru je ime s kojim će Hibernate mapirati vrijednost.

4.7.3. Hibernate generirani SQL za kompleksan upit

Hibernate sam generira SQL, primjer se može vidjeti u kodu 31.

```

1.      select
2.          distinct r1_0.id_radnik,
3.          r1_0.radnik_nadred_fk,
4.          r1_0.ime,
5.          r1_0.prezime,
6.          r1_0.oib,
7.          sf1_0.oznaka,
8.          r1_0.datum_rod,
9.          rvf1_0.naziv,
10.         ro1_0.odjel_fk,
11.         ur1_0.id Ugovor_rad,
12.         ro1_0.datum_od,
13.         ur1_0.datum_od
14.      from
15.         radnik r1_0
16.      left join
17.         spol sf1_0
18.         on sf1_0.id_spol=r1_0.spol_fk
19.      left join
20.         radnik_vrsta rvf1_0
21.         on rvf1_0.id_radnik_vrsta=r1_0.radnik_vrsta_fk
22.      left join
23.         radnik_odjel ro1_0
24.         on r1_0.id_radnik=ro1_0.radnik_fk
25.      left join
26.         ugovor_rad ur1_0
27.         on r1_0.id_radnik=ur1_0.radnik_fk
28.      where
29.         r1_0.id_radnik=?
30.      order by
31.         1 desc,
32.         11 desc,
33.         12 desc

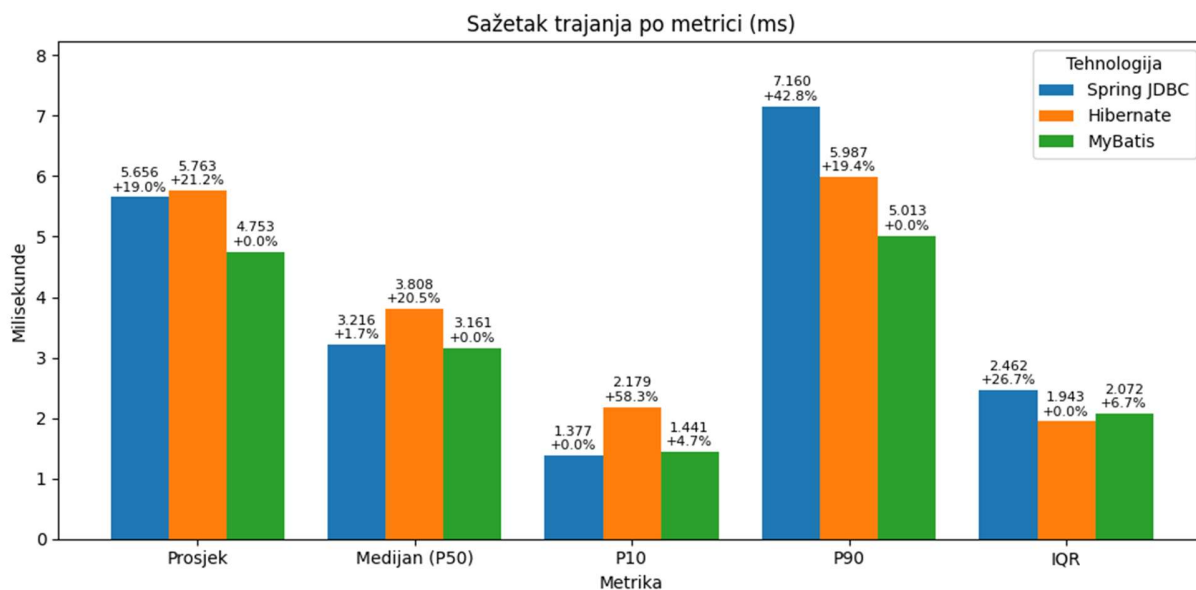
```

Kod 31. SQL generiran Hibernatom

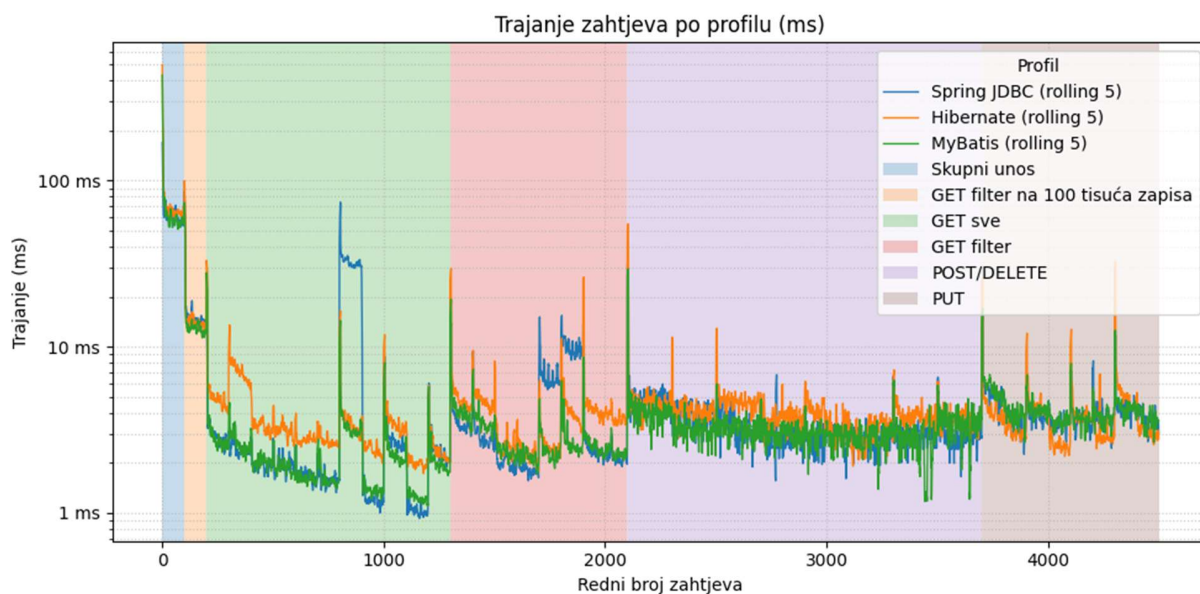
Generirani SQL je čitljiv ali nije na istoj razini čitljivosti koju bi ljudska osoba postigla. To je zato što se SQL kod Hibernate ne bi ni trebao često čitati, ali zbog toga pomalo otežava *debugiranje*.

4.8. Performansno testiranje

Performansno testiranje vršilo se tako da se u Java aplikaciji mjeri vrijeme od preuzetog upita na *endpointu* do odgovora na *endpointu*. Ovim putem istovremeno je testirana kvaliteta SQL upita i same performanse tehnologije potrebne da se daj SQL upit izvrši. Svaki *endpoint* koji ulazi u testiranje bio je pozvan 100 puta. U grafu 1. i 2. prikazana je usporedba svih napravljenih upita.



Graf 1. Usporedba tehnologija prema metrikama (svi podaci)



Graf 2. Usporedba trajanja upita tehnologija kroz broj upita

- Iz grafa 1. može se izvući krivi zaključak da je Mybatis najbolji po performansama zahtjeva, ali ako se pogleda graf 2. može se primijetiti da svaka tehnologija u nekom slučaju najbolja.
- Razlog za veliki skok Spring JDBC trajanja upita za vrijeme GET sve i GET filter leži u tome da automatski *mapper* podataka loše radi kada naiđe na BYTEA podatak iz baze.
- Hibernate u nekim slučajevima zaostaje kada se koristi GET sa dohvatu svih podataka i kada se koristi GET za dohvat filtriranih podataka. Hibernate ovisi koristi li se Criteria, JPQL, ili defaultni Hibernate. Korištenje JPQL povećava performanse, kao i ne korištenje *@Entity* anotacije kako Hibernate ne bi trebao provjeravati vezu između tablica za vrijeme upita.
- Hibernate ima najbolje performanse kod PUT upita dok su za ostale upite Spring JDBC većinom podjednaki.

Detaljniji grafovi i podaci se mogu pronaći u [prilogu 1.](#)

4.9. Usporedna analiza nakon implementacije

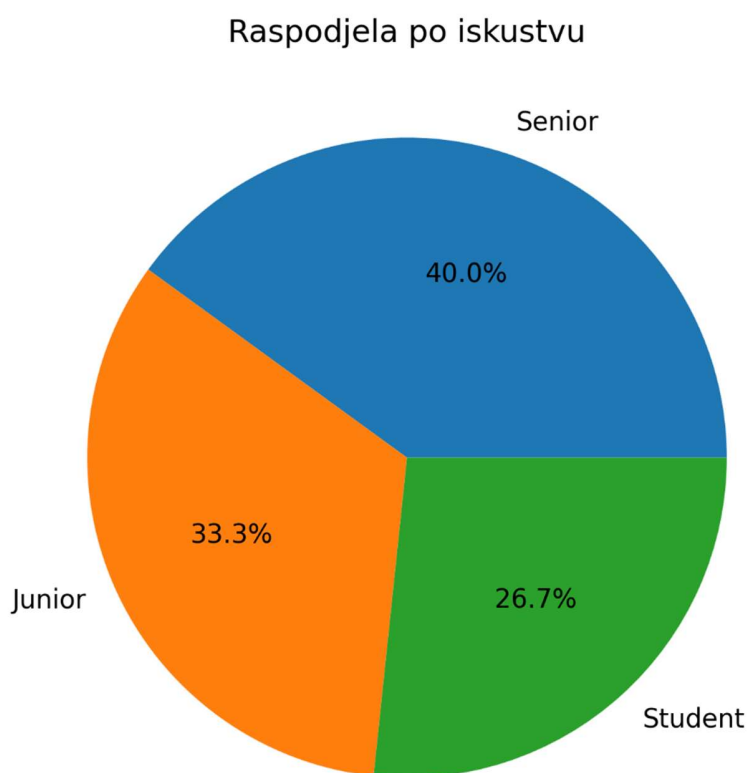
Tablica 1. Usporedna analiza tehnologija

Značajka / Tehnologija	Spring JDBC	Hibernate (JPA)	MyBatis
Tip pristupa	Polu-automatski, ručno pisanje SQL-a	Potpuno automatski ORM ili ručno	Polu-automatski, SQL u XML ili anotacijama
Pisanje SQL-a	Ručno	Automatski generiran od strane ORM-a, ručno kod kompleksnih upita	Ručno
Mapiranje objekata na SQL upit	Ručno ili automatsko s parametrima u SQL-u	Automatsko s parametrima, u kompleksnim upitima ručno s parametrima	Automatsko s parametrima u XML-u ili anotacijama
Mapiranje rezultata u Java objekte	Automatsko	Automatsko	Automatsko
Krivulja učenja	Srednja do visoka	Niska do visoka	Srednja do visoka

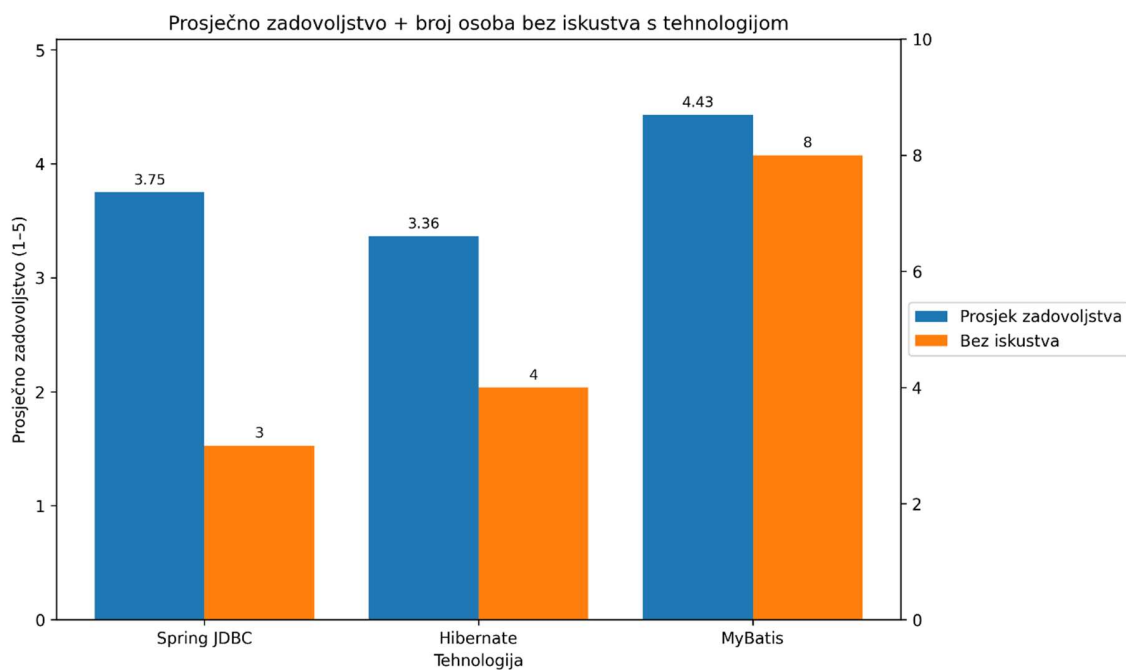
Fleksibilnost upita	Najveća – potpuna kontrola nad SQL-om i rezultatima	Ograničena (JPQL, Criteria API ili JPQL)	Visoka – pun SQL podržan i dodatni načini pisanja
Performanse	Odlične, ovise o kvaliteti ručno napisanog SQL	Malo lošije i ovise o načinu na koji je napisan upit	Odlične, ovise o kvaliteti ručno napisanog SQL
Skupni unos	Da	Da	Ručna implementacija
Transakcije	Da	Da	Da
Prednosti	○ Kontrola nad SQL-om ○ Mogućnost prilagođene implementacije ○ Kontrola nad rezultatima	○ Manje pisanja ○ Laka migracija baze podataka ○ Mogućnost kontrole i izrade modela baze	○ Kontrola nad SQL-om ○ Logika na razini SQL-a
Nedostaci	○ Veći rizik od grešaka u SQL-u ○ Potrebno više prilagođene implementacije	○ Rizik neželjenih upita ○ Teže <i>debuggiranje</i> ○ Za potpunu funkcionalnost potrebno imati model baze u kodu ○ Malo lošije performanse	○ Pisanje logike i SQL-a u XML-u ○ Veći rizik od grešaka u SQL-u ○ Potrebno je sve ručno implementirati
Idealno za		Male projekte ili velike projekte s puno entiteta	Projekte s kompleksnim SQL upitima

5. Istraživanje i anketa

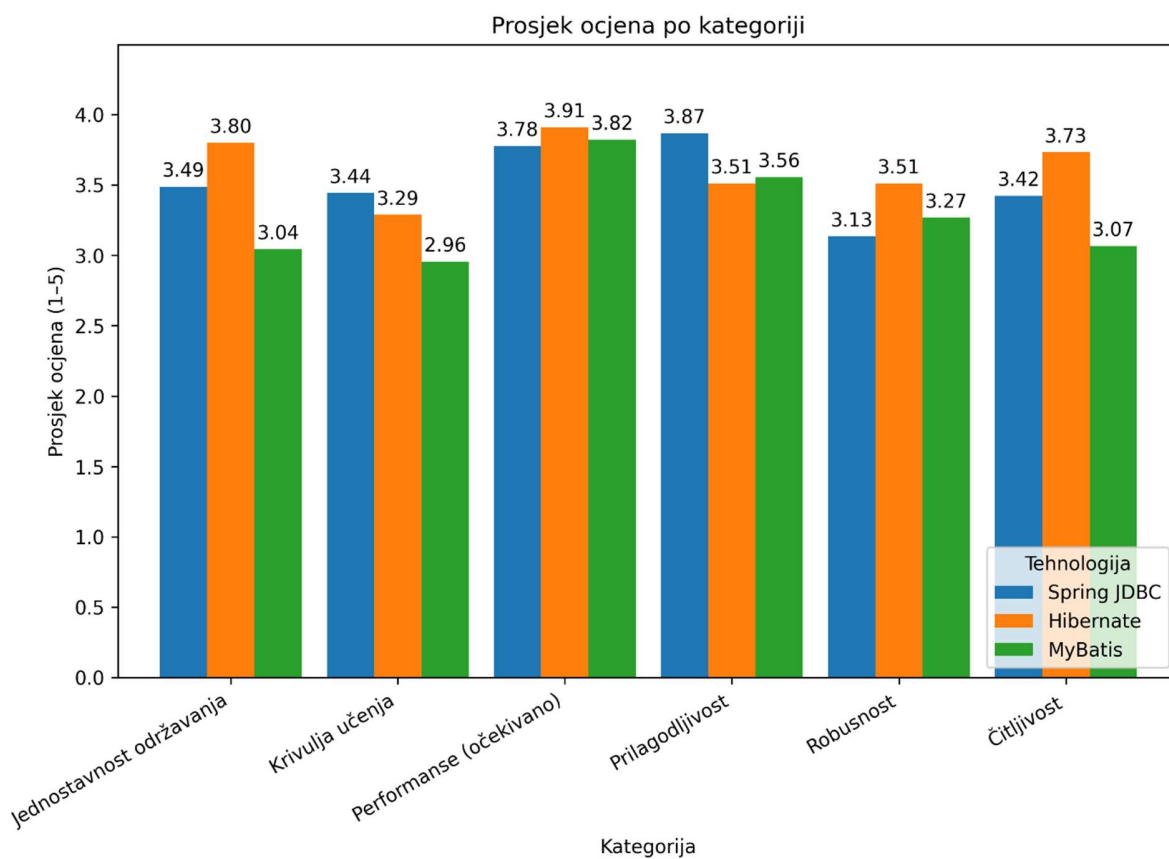
Anonimna anketa je provedena putem Google forms te se pitanja i podaci mogu pronaći u [prilogu](#) 1. Cilj ankete je bio prikupljanje mišljenja korisnika ovih tehnologija, te koliko se koja tehnologija koristi. Anketu je ispunilo 15 osoba. Na grafu 3. prikazana je distribucija po iskustvu, na grafu 4. prikazano je zadovoljstvo osoba pri korištenju tehnologija te na grafu 5. je prikazana usporedba srednjih ocjena svake tehnologije.



Graf 3. Raspodjela ankete po iskustvu



Graf 4. Prosječno zadovoljstvo s tehnologijama



Graf 5. Prosječne ocjene svih kategorija i pitanja po tehnologiji

- Pogledom na graf 5., uočljivo je da većina osoba misli da je Hibernate najčitljivija, najrobusnija i tehnologija koju je najlakše održavati, a Mybatis najlošija u ovim kategorijama, osim robusnosti gdje je Spring JDBC najlošija.
- U krivulji učenja i prilagodljivosti Spring JDBC ima najveću ocjenu, dok Mybatis ima najmanju u krivulji učenja a Hibernate najmanju u prilagodljivosti.
U kategoriji očekivanih performansi sve tehnologije imaju sličnu ocjenu. Hibernate ima najveću a Spring JDBC najmanju.

6. Zaključak

U ovom završnom radu prikazane i uspoređene su 3 tehnologije za komunikaciju s bazom podataka u okviru Java Spring Boot aplikacije: Spring JDBC, Hibernate ORM i Mybatis persistence okvir. Analizirane su njihove prednosti performanse i nedostaci popraćeni s realnim implementacijama: jednostavnih, složenih, ugniježđenim, transakcijama, skupnim unosima i mapiranja objekata.

Rezultati su pokazali da niti jedna tehnologija nije najbrža u svim slučajevima, već se svaka istaknula u određenim upitima. Hibernate se pokazao boljim kod složenijih operacija poput ažuriranja, dok je Spring JDBC imao prednost kod jednostavnijih scenarija uz punu kontrolu nad SQL-om. MyBatis je zauzeo srednje mjesto, nudeći dobru fleksibilnost i odlične performanse kod ostalih kompleksnih upita, ali uz dodatno održavanje i ručno pisanje SQL-a.

Provedena anketa među programerima pokazala je da je Hibernate-ov pristup kreiranju upita poželjan ali i nosi najmanje zadovoljstvo prošlih iskustva. Spring JDBC pokazao se kao snažan odabir mnoštva programera te se MyBatis čini kompliciran ali nosi veliko zadovoljstvo.

Izbor tehnologije ovisi o zahtjevima projekta. Na temelju provedene usporedbe može se zaključiti da je Hibernate najbolji za brzi razvoj i veću apstrakciju, za projekte koji zahtijevaju maksimalnu kontrolu i performanse preporučuje se Spring JDBC, dok MyBatis zauzima mjesto u slučajevima gdje se upiti ne žele apstrahirati zbog performansi, ali opet imat na raspolaganju dodatne alate.

Ovim radom naglašena je važnost kreiranja praktičnog primjera prije donošenja odluke koju tehnologiju koristiti. Dobiveni rezultati i kreirani kod dostupan u 1. može poslužiti kao smjernica budućim programerima za odabir najprikladnije tehnologije za komunikaciju između aplikacije i baze podataka.

7. Literatura

- [1] PostgreSQL Global Development Group, »PostgreSQL 17 Documentation,« 2024. [Mrežno]. Available: <https://www.postgresql.org/files/documentation/pdf/17/postgresql-17-A4.pdf>. [Pokušaj pristupa 8 7 2025].
- [2] S. Salunke i A. Ouda, »A Performance Benchmark for the PostgreSQL and MySQL Databases,« 19 10 2024. [Mrežno]. Available: <https://www.mdpi.com/1999-5903/16/10/382>. [Pokušaj pristupa 8 7 2025].
- [3] DBeaver Community, »DBeaver User Guide,« DBeaver Community, 2025. [Mrežno]. Available: <https://dbeaver.com/docs/dbeaver/>. [Pokušaj pristupa 8 7 2025].
- [4] Oracle, »Java Platform, Standard Edition 8 – Language Guide,« 2014. [Mrežno]. Available: <https://docs.oracle.com/javase/8/docs/technotes/guides/index.html>. [Pokušaj pristupa 8 7 2025].
- [5] Computer Weekly, »Write once, run anywhere?,« Computer Weekly, 2 5 2002. [Mrežno]. Available: <https://web.archive.org/web/20210813193857/https://www.computerweekly.com/feature/Write-once-run-anywhere>. [Pokušaj pristupa 8 7 2025].
- [6] Tiobe Software, »TIOBE Index – Java Programming Language,« Tiobe Software, 2025. [Mrežno]. Available: <https://www.tiobe.com/tiobe-index/>. [Pokušaj pristupa 8 7 2025].
- [7] Project Lombok, »Project Lombok,« Project Lombok, 2025. [Mrežno]. Available: <https://projectlombok.org>. [Pokušaj pristupa 8 7 2025].
- [8] Project Lombok, »var • Lombok Features,« Project Lombok, 2025. [Mrežno]. Available: <https://projectlombok.org/features/var>. [Pokušaj pristupa 8 7 2025].
- [9] W3Schools.com, »Java var Keyword,« W3Schools, 2025. [Mrežno]. Available: https://www.w3schools.com/java/ref_keyword_var.asp. [Pokušaj pristupa 8 7 2025].

- [10 Spring Projects, »spring-boot,« Spring Projects, 2025. [Mrežno]. Available:
] <https://github.com/spring-projects/spring-boot>. [Pokušaj pristupa 8 7 2025].
- [11 S. Team, »Spring Framework,« Spring Team, 2025. [Mrežno]. Available:
] <https://spring.io/projects/spring-framework>. [Pokušaj pristupa 8 7 2025].
- [12 Spring Team, »Spring Boot,« Spring Team, 2025. [Mrežno]. Available:
] <https://spring.io/projects/spring-boot>. [Pokušaj pristupa 8 7 2025].
- [13 Spring Team, »2.2.1. Bean container basics,« Spring Team, 2025. [Mrežno]. Available:
] <https://docs.spring.io/spring-framework/reference/core/beans/basics.html>. [Pokušaj pristupa 8 7 2025].
- [14 Spring Team, »7. Aspect Oriented Programming,« Spring Team, 2025. [Mrežno]. Available:
] <https://docs.spring.io/spring-framework/reference/core/aop.html>. [Pokušaj pristupa 8 7 2025].
- [15 Spring Team, »2.5. Dependencies Between Beans,« Spring Team, 2025. [Mrežno].
] Available: <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>. [Pokušaj pristupa 8 7 2025].
- [16 Spring Team, »16. Spring MVC,« Spring Team, 2025. [Mrežno]. Available:
] <https://docs.spring.io/spring-framework/reference/web/webmvc.html>. [Pokušaj pristupa 8 7 2025].
- [17 R. Fielding, »Architectural Styles and the Design of Network-based Software Architectures,« University of California, Irvine, 2000. [Mrežno]. Available:
https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. [Pokušaj pristupa 8 7 2025].
- [18 Spring Team, »Spring Boot DevTools,« Spring Team, 2025. [Mrežno]. Available:
] <https://web.archive.org/web/20250719064102/https://docs.spring.io/spring-boot/reference/using/devtools.html>. [Pokušaj pristupa 19 7 2025].

- [19 SmartBear Software, »Swagger Docs,« SmartBear Software, 2025. [Mrežno]. Available:
] https://web.archive.org/web/20250719070800/https://swagger.io/docs/specification/v3_0/about/. [Pokušaj pristupa 19 7 2025].
- [20 Spring Team, »Data Access with JDBC :: Spring framework,« Spring Team, 2025.
] [Mrežno]. Available:
<https://web.archive.org/web/20250719050115/https://docs.spring.io/spring-framework/reference/data-access/jdbc.html>. [Pokušaj pristupa 19 7 2025].
- [21 Red Hat, Inc., »Hibernate ORM,« Red Hat, Inc., [Mrežno]. Available:
] <https://web.archive.org/web/20250719051458/https://hibernate.org/orm/>. [Pokušaj pristupa 19 7 2025].
- [22 MyBatis.org, »MyBatis 3 – Introduction,« MyBatis.org, 2025. [Mrežno]. Available:
] <https://web.archive.org/web/20250716071805/https://mybatis.org/mybatis-3/index.html>.
[Pokušaj pristupa 19 7 2025].

8. Prilozi

1. Link na Github repozitorij u kojem se nalaze svi materijali, kodovi i rezultati završnog rada: <https://github.com/SaricLuka/hrsustav>

Repozitorij sadrži subrepos direktorij u kojem su linkovi na razvrstane dijelove projekta. Api je kod glavne aplikacije. Tools je python kod koji se koristio za testiranje i kreiranje grafova. Docs sadrži sve podatke, grafove i dokumente završnog rada.